

Thesis progress report

LLM-Guided Neural Network Compilation for Real-time UAV Edge Inference

Atul Kumar Gupta · 25MCSS06

Why the companion computer is a compile problem

A small drone splits the work in two. PX4, on the flight controller, keeps the aircraft in the air. A Linux computer beside it has to run the camera network, and it has to finish before the next frame is due.

That second computer is the companion. It is where people, vehicles, a landing patch, or a depth cue would be recognised. The flight controller does not run those networks. If the companion is late, the aircraft is still flying, but it is flying without the picture.

Turning a trained network into something the companion can run is a compile recipe. The recipe chooses which operations may be joined, whether the weights stay in 32-bit or drop to 8-bit, and how many CPU threads are used. In practice that recipe is still picked by habit. A choice that is fast on a desktop GPU is often slow on a four-core ARM CPU, and an 8-bit version can be both faster and numerically wrong.

The lab tools around that computer had to be named before any compiler was written. PX4 is the flight stack, in the air and in simulation. Gazebo is the simulated world. ROS 2 is the message bus: programs publish and subscribe on topics, they do not call each other directly. ONNX is the file that holds a trained network as a graph of operations and tensors, which is the object a compiler can inspect.

A camera that produces a frame about every 33 ms does not leave a large budget. If the network and the recipe around it take longer than that, frames are dropped. The recipe is therefore part of the research problem, not a setup step that can be left to taste.

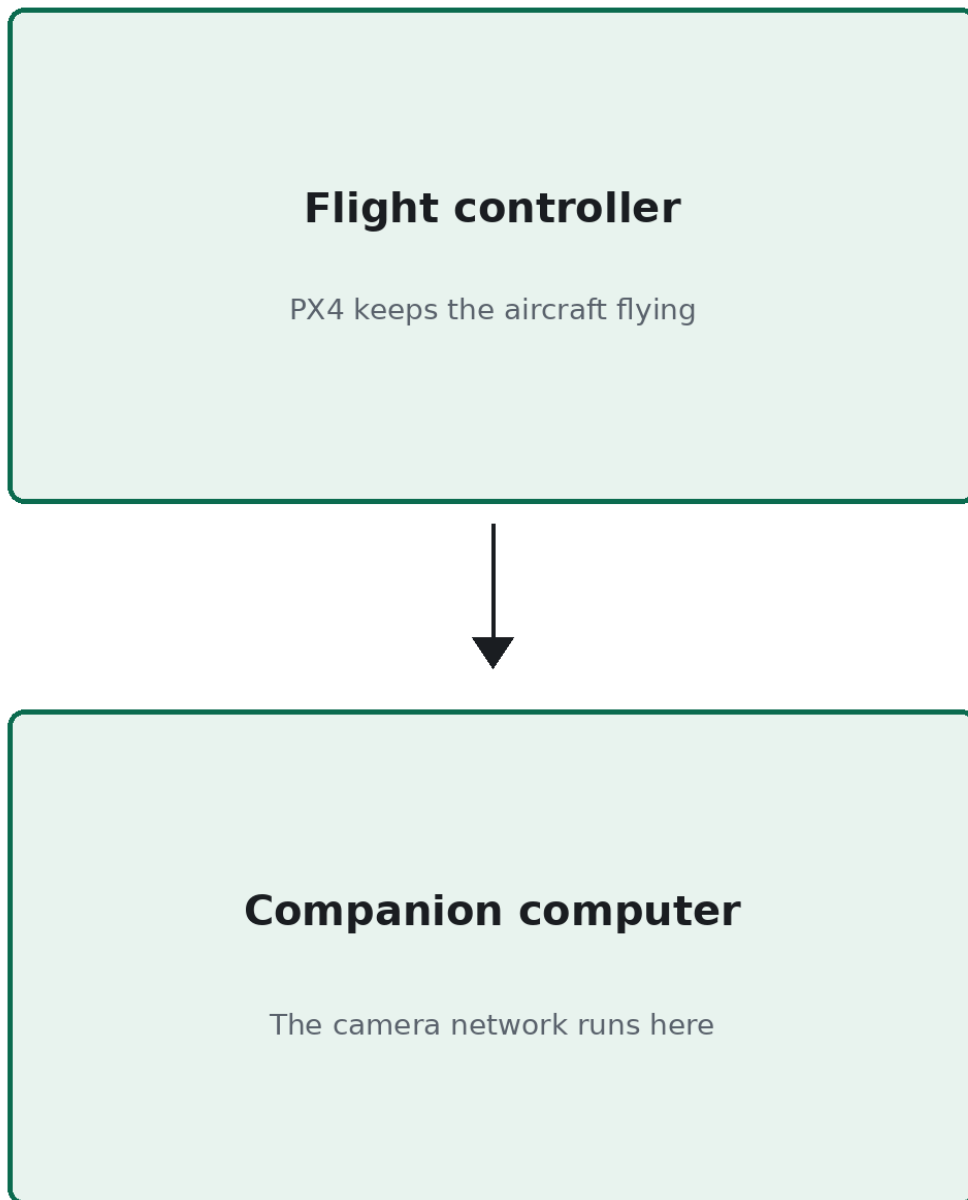


Figure 1. PX4 flies. The companion computer is the only place the camera network runs.

Flight and vision are separate. What is still open is who is allowed to choose the compile recipe.